Spectacles: A Verified Compiler from Semiring-Weighted Automata to Zero-Knowledge Scoring Circuits

Abstract

We present SPECTACLES, a system that produces the first *contamination-certified evaluation certificates*: machine-checkable proofs that a benchmark score is correct *and* that the test data was not leaked into training. The key insight is that the NLP scoring functions used in practice—exact match, BLEU, ROUGE, token F1, regex match, and pass@k—decompose into weighted finite automata (WFA) over semirings. SPECTACLES compiles these WFAs to STARK arithmetic circuits via a two-tier architecture: an *algebraic* tier for semirings that embed into the Goldilocks field $\mathbb{F}_p$ ($p = 2^{64} - 2^{32} + 1$) via injective homomorphisms, and a *gadget-assisted* tier for non-embeddable semirings such as the tropical semiring $(\mathbb{Z} \cup \{+\infty\}, \min, +)$, which requires bit-decomposition comparison gadgets. SPECTACLES composes the evaluation proof with an OPRF-based private set intersection (PSI) protocol, yielding a single certificate that bounds n-gram overlap between the training corpus and the test set below a threshold τ without revealing either dataset. The system further provides decidable metric equivalence checking via WFA minimization and coalgebraic bisimulation, enabling auditors to verify that two implementations of the same metric compute identical functions. We implement SPECTACLES in Rust, including a triple implementation of each metric (reference algorithm, WFA execution, and circuit witness), validated by differential testing over 100 K random input pairs. SPECTACLES is, to our knowledge, the first verified zero-knowledge compiler for a specification language targeting NLP evaluation metrics.

1 Introduction

The evaluation of large language models (LLMs) faces two simultaneous trust crises. First, *score integrity*: benchmark scores are computed by imperative Python scripts that offer no machine-checkable correctness argument. A subtle implementation bug—an off-by-one in n-gram clipping, a misplaced tokenization boundary—can silently inflate or deflate scores, and no existing infrastructure detects such errors after the fact [17]. Second, *contamination*: model providers may, inadvertently or deliberately, train on test data. Statistical detection methods [19, 21] produce population-level estimates but cannot certify individual evaluations, and they break down when the overlap is small or the model memorizes paraphrases rather than verbatim copies.

Why existing approaches fall short. VerifiableEvals [22] applies zero-knowledge proofs to evaluation, but proves that a *specific program* was executed correctly—it certifies program *execution*, not specification *conformance*. If the program contains a bug, the proof faithfully certifies the buggy answer. Moreover, it offers no contamination guarantee whatsoever.

The WFA insight. We observe that every standard NLP scoring function decomposes into a *weighted finite automaton* (WFA) over an appropriate semiring. Exact match is a Boolean-semiring acceptor; BLEU [16] decomposes into four counting-semiring WFAs (one per n-gram order) followed by a geometric-mean aggregation gadget; ROUGE-L [14] is a tropical-semiring WFA computing a longest common subsequence. This algebraic structure is the fulcrum of our approach: it simultaneously provides (i) a *formal semantics* for the metric, rooted in the well-studied theory of weighted automata [20, 4], and (ii) a *compilation target* whose transition structure maps directly to STARK arithmetic constraints.

The Spectacles approach. Spectacles is a six-layer pipeline:

$$\text{EvalSpec DSL} \xrightarrow{\text{parse}} \text{WFA} \xrightarrow{\text{compile}} \mathsf{AIR} \xrightarrow{\text{prove}} \mathsf{STARK}\ \text{proof} \xrightarrow{\text{compose}} \text{Certificate}$$

The EvalSpec domain-specific language lets metric designers specify scoring functions in a typed, semiring-annotated notation. The Spectacles compiler lowers an EvalSpec program to a WFA, compiles the WFA to an Algebraic Intermediate Representation (AIR) over the Goldilocks field, and invokes a STARK prover. The resulting proof is composed with a PSI-based contamination check to yield a single *contamination-certified evaluation certificate*.

Contributions.

1. The first verified zero-knowledge compiler for a specification language (EvalSpec) targeting NLP evaluation metrics (§3–§5).
2. A two-tier compilation strategy—algebraic for embeddable semirings, gadget-assisted for tropical—that covers all seven standard metrics with a single framework (§5).
3. Contamination-certified evaluation via composition of STARK proofs with OPRF-based PSI, including a trie-structured optimization that reduces communication from $O(|L_1| \cdot |L_2| \cdot \lambda)$ to $O((n_1 + n_2) \cdot \lambda)$ (§6).
4. Decidable metric equivalence checking via WFA minimization and coalgebraic bisimulation (§7).
5. A Rust implementation including triple implementations of every metric validated by differential testing (§8, §9).

2 Background

2.1 Weighted Finite Automata over Semirings

Definition 2.1 (Semiring)**.** *A* semiring *is a tuple* $(\mathcal{S}, \oplus, \otimes, \bar{0}, \bar{1})$ *where* $(\mathcal{S}, \oplus, \bar{0})$ *is a commutative monoid,* $(\mathcal{S}, \otimes, \bar{1})$ *is a monoid,* $\otimes$ *distributes over* $\oplus$*, and* $\bar{0}$ *annihilates:* $\bar{0} \otimes s = s \otimes \bar{0} = \bar{0}$ *for all* $s \in \mathcal{S}$*.*

Table 1 lists the semirings used in Spectacles.

Definition 2.2 (Weighted Finite Automaton)**.** *A* weighted finite automaton *over a semiring* $(\mathcal{S}, \oplus, \otimes, \bar{0}, \bar{1})$ *and alphabet* Σ *is a tuple* $\mathcal{A} = (Q, \Sigma, \{\mu_\sigma\}_{\sigma \in \Sigma}, \alpha, \eta)$ *where* Q *is a finite set of states, each* $\mu_\sigma \in \mathcal{S}^{|Q| \times |Q|}$ *is a transition matrix,* $\alpha \in \mathcal{S}^{1 \times |Q|}$ *is the initial weight vector, and* $\eta \in \mathcal{S}^{|Q| \times 1}$ *is the final weight vector. The weight assigned to a word* $w = \sigma_1 \cdots \sigma_n \in \Sigma^*$ *is*

$$\mathcal{A}(w) \;=\; \alpha \;\otimes\; \mu_{\sigma_1} \;\otimes\; \cdots \;\otimes\; \mu_{\sigma_n} \;\otimes\; \eta\,. \tag{1}$$

The formal power series $\mathcal{A} : \Sigma^* \to \mathcal{S}$ recognized by a WFA is a central object in algebraic automata theory [20, 4, 6].

Table 1: Semirings supported by SPECTACLES.

Name	$\mathcal{S}$	$\oplus$	$\otimes$	$\bar{0}$	$\bar{1}$
Boolean	$\{0,1\}$	$\vee$	$\wedge$	0	1
Counting	$\mathbb{N}$	$+$	$\cdot$	0	1
Tropical	$\mathbb{Z} \cup \{+\infty\}$	min	$+$	$+\infty$	0
Real	$\mathbb{R}$	$+$	$\cdot$	0	1
MaxPlus	$\mathbb{R} \cup \{-\infty\}$	max	$+$	$-\infty$	0
Viterbi	$[0,1]$	max	$\cdot$	0	1
Goldilocks	$\mathbb{F}_p$	$+_p$	$\cdot_p$	0	1

2.2 STARK Proof System

A Scalable Transparent ARgument of Knowledge (STARK) [3] enables a prover to convince a verifier of the correctness of a computation without revealing the computation's input. The computation is expressed as an *Algebraic Intermediate Representation* (AIR): a set of polynomial constraints $\{C_i(x_0, \ldots, x_{w-1}, x'_0, \ldots, x'_{w-1})\}_{i=1}^{c}$ over w columns ("registers") and T rows ("time steps") of an *execution trace* $\mathsf{Tr} \in \mathbb{F}_p^{T \times w}$. Transition constraints assert relationships between consecutive rows; boundary constraints pin values at specific rows. Soundness relies on the Fast Reed–Solomon Interactive Oracle Proof of Proximity (FRI) [2], which tests proximity to the Reed–Solomon code via recursive halving.

SPECTACLES targets the *Goldilocks field* $\mathbb{F}_p$ with $p = 2^{64} - 2^{32} + 1$, chosen because (i) field elements fit in a 64-bit machine word, (ii) $p - 1 = 2^{32} \cdot (2^{32} - 1)$ admits a multiplicative subgroup of order 2^{32}, enabling efficient FFT-based polynomial operations, and (iii) the reduction $x \bmod p$ requires only a shift and subtraction, avoiding generic division.

2.3 Private Set Intersection

An *oblivious pseudo-random function* (OPRF) allows a client holding input x and a server holding key k to jointly compute $F_k(x)$ such that the client learns $F_k(x)$ and nothing about k, while the server learns nothing about x. OPRF-based *private set intersection* (PSI) [7] enables two parties to compute the intersection of their sets without revealing elements outside the intersection. In the PSI-cardinality variant, the parties learn only $|S_1 \cap S_2|$, and in threshold mode, they learn only whether $|S_1 \cap S_2| < \tau$ for a predetermined threshold τ.

3 System Architecture

SPECTACLES is organized as a six-layer pipeline (Figure 1). Each layer has a well-defined input/output interface, and layers communicate via serializable intermediate representations.

Layer 1: EvalSpec DSL. The specification layer provides a typed, semiring-annotated domain-specific language for defining evaluation metrics (§4). The parser produces a typed abstract syntax tree (AST) annotated with the inferred semiring type.

Layer 2: WFA Engine. The automata layer implements weighted finite automata over arbitrary semirings. It provides construction, product, composition, minimization [10], and equivalence

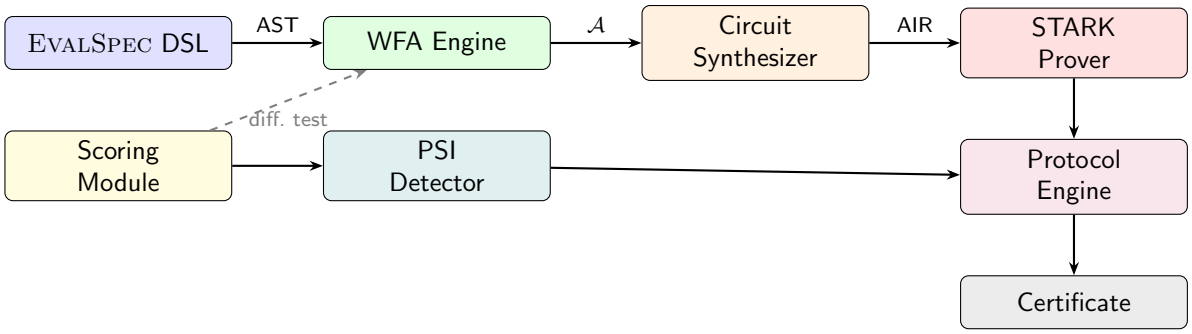

Figure 1: Architecture of Spectacles. Solid arrows show the compilation and certification pipeline; the dashed arrow indicates differential testing between the scoring module and the WFA engine.

checking via coalgebraic bisimulation. The engine also performs field embedding: mapping semiring elements to the Goldilocks field via injective homomorphisms (§5).

Layer 3: Circuit Synthesizer. The synthesizer compiles a WFA into an AIR program. Each WFA transition μ_σ becomes a matrix–vector multiply constraint over $\mathbb{F}_p$. The synthesizer handles two tiers: algebraic (direct embedding) and gadget-assisted (bit-decomposition for tropical min). It also generates the execution trace from a concrete input pair.

Layer 4: STARK Prover/Verifier. The STARK layer implements the FRI-based interactive oracle proof protocol. The prover commits to the execution trace via Merkle trees over BLAKE3 hashes, performs low-degree extensions, and responds to verifier challenges. The verifier checks Merkle paths, evaluates constraints at queried points, and verifies FRI consistency.

Layer 5: Protocol Engine. The protocol engine implements a commit–reveal–verify state machine that orchestrates the full evaluation protocol. A transcript records every message exchanged, and the final certificate bundles the STARK proof, PSI result, and protocol metadata.

Layer 6: PSI Contamination Detector. The contamination module implements OPRF-based private set intersection with a trie-structured optimization that exploits n-gram prefix sharing. In threshold mode, the protocol produces a proof that the n-gram overlap between the training corpus and the test set is below τ without revealing the count or the overlapping n-grams.

4 The EvalSpec DSL

EvalSpec is a typed functional language for specifying NLP evaluation metrics. Its design goals are (i) expressiveness sufficient for the seven standard metrics, (ii) decidable type inference with automatic semiring selection, and (iii) a denotational semantics that maps every well-typed program to a WFA.

4.1 Syntax

An EvalSpec program consists of a sequence of metric declarations. Each declaration names the metric, lists its parameters, specifies (or allows inference of) the semiring, and provides a body

expression:

```
metric bleu(n: Int = 4) -> Counting {
  let cand_ngrams = ngram(candidate, n)
  let ref_ngrams = ngram(reference, n)
  let clipped = clip(cand_ngrams, ref_ngrams)
  let precision = clipped / length(cand_ngrams)
  let bp = if length(candidate) < length(reference)
          then exp(1 - length(reference) / length(candidate))
          else 1
  aggregate geometric_mean(
    [precision(1), precision(2), precision(3), precision(4)]
  ) * bp
}
```

Listing 1: BLEU-4 in EvalSpec.

The expression language includes let-bindings, conditionals, pattern matching, n-gram extraction (`ngram`), tokenization (`tokenize`), automaton composition (`compose`), bounded counting (`clip`), and aggregation operators (`aggregate sum`, `aggregate geometric_mean`).

4.2 Formal Grammar

EvalSpec has a formally specified BNF grammar (see supplementary material). The core productions are:

$$\begin{aligned}
\langle\text{program}\rangle &::= \langle\text{decl}\rangle^* \\
\langle\text{decl}\rangle &::= \texttt{metric}\ \langle\text{id}\rangle\texttt{(}\langle\text{params}\rangle\texttt{)->}\ \langle\text{semiring}\rangle\ \{\langle\text{expr}\rangle\} \\
\langle\text{expr}\rangle &::= \langle\text{lit}\rangle \mid \langle\text{var}\rangle \mid \langle\text{expr}\rangle\ \langle\text{binop}\rangle\ \langle\text{expr}\rangle \mid \texttt{ngram}(\langle\text{expr}\rangle, n) \mid \cdots
\end{aligned}$$

The full grammar is implemented as a hand-written recursive descent parser (3,751 LoC) with Pratt-parser precedence handling.

4.3 Type System and Semiring Inference

The type system assigns each expression a pair $(\tau, \mathcal{S})$ consisting of a base type τ (string, integer, float, Boolean, list, record) and a semiring annotation $\mathcal{S}$. Semiring inference propagates bottom-up: `ngram` produces a counting semiring; `clip` preserves its input semiring; the Boolean operators $\wedge$, $\vee$ produce the Boolean semiring; the `lcs` (longest common subsequence) combinator produces the MaxPlus semiring, which the compiler maps to tropical.

Definition 4.1 (Semiring inference rules (selected))**.**

$$\frac{\Gamma \vdash e : (\mathit{List}(\mathit{String}),\, \mathcal{S})}{\Gamma \vdash \mathit{ngram}(e, n) : (\mathit{List}(\mathit{String}),\, \mathsf{Counting})} \qquad \frac{\Gamma \vdash e_1 : (\tau,\, \mathcal{S}_1) \quad \Gamma \vdash e_2 : (\tau,\, \mathcal{S}_2)}{\Gamma \vdash e_1 \oplus e_2 : (\tau,\, \mathcal{S}_1 \sqcup \mathcal{S}_2)} \tag{2}$$

where $\sqcup$ is the join in the semiring lattice: Boolean $\sqsubseteq$ Counting $\sqsubseteq$ Real, and Tropical is incomparable.

4.4 Denotational Semantics

Every well-typed EvalSpec expression e of type $(\tau, \mathcal{S})$ denotes a function $e : \Sigma^* \times \Sigma^* \to \mathcal{S}$ from (candidate, reference) string pairs to semiring values. The key semantic clause is:

Table 2: Built-in metrics in EvalSpec with semiring annotations and WFA state complexity.

Metric	Semiring	WFA States	Tier	Reference
Exact Match	Boolean	$O(n)$	1	—
Token F1	Counting	$O(\|V\|)$	1	—
BLEU	Counting	$O(n^4)$ decomp.	1+gadget	[16]
ROUGE-N	Counting	$O(n^k)$	1	[14]
ROUGE-L	MaxPlus/Tropical	$O(nm)$	2	[14]
Regex Match	Boolean	$O(2^k)$	1	—
Pass@k	Counting	$O(kt)$	1	[5]

$$\texttt{metric}\ m(\vec{p})\ \texttt{->}\ \mathcal{S}\ \{e\}\ =\ \lambda\,(cand, ref).\ e_{\Gamma[\texttt{candidate}\mapsto cand,\ \texttt{reference}\mapsto ref]} \tag{3}$$

The compiler constructs a WFA $\mathcal{A}_e$ such that $\mathcal{A}_e(w) = e(w)$ for all inputs w. Correctness of this translation is the central theorem of §5.

4.5 Built-in Metrics

Table 2 lists the seven built-in metrics, their semiring annotations, and their WFA state complexity.

5 Two-Tier Compilation

The Spectacles compiler translates a WFA $\mathcal{A} = (Q, \Sigma, \{\mu_\sigma\}, \alpha, \eta)$ over semiring $\mathcal{S}$ into an AIR program over the Goldilocks field $\mathbb{F}_p$. The translation depends on whether $\mathcal{S}$ admits an injective semiring homomorphism into $\mathbb{F}_p$.

5.1 Tier 1: Algebraic Compilation

Definition 5.1 (Embeddable semiring)**.** *A semiring* $(\mathcal{S}, \oplus, \otimes, \bar{0}, \bar{1})$ *is* $\mathbb{F}_p$-embeddable *if there exists an injective map* $\phi : \mathcal{S} \to \mathbb{F}_p$ *satisfying*

$$\phi(\bar{0}) = 0, \quad \phi(\bar{1}) = 1, \quad \phi(a \oplus b) = \phi(a) + \phi(b), \quad \phi(a \otimes b) = \phi(a) \cdot \phi(b). \tag{4}$$

The Boolean, counting, bounded counting, and real semirings are all $\mathbb{F}_p$-embeddable (the Boolean semiring via $\phi(\mathsf{false}) = 0$, $\phi(\mathsf{true}) = 1$; the counting semiring via the natural inclusion $\mathbb{N} \hookrightarrow \mathbb{F}_p$, valid for inputs smaller than p). Note that the Boolean embedding maps $\otimes = \wedge$ to field multiplication, but $\oplus = \vee$ does not map to field addition in general ($\vee$ is idempotent: $1 \vee 1 = 1$, but $1 + 1 = 2$). The embedding is valid for deterministic WFA execution, where at most one transition per state is active per step, so the idempotency mismatch never arises.

For an embeddable semiring, the compilation is straightforward. Let $|Q| = n$. The AIR program has $2n + O(1)$ columns and one row per input symbol.

Trace layout. The execution trace $\mathsf{Tr} \in \mathbb{F}_p^{T \times w}$ has columns:

- $\mathbf{s}_0, \ldots, \mathbf{s}_{n-1}$: state weight vector at the current step.
- $\mathbf{s}'_0, \ldots, \mathbf{s}'_{n-1}$: state weight vector at the next step.
- σ: input symbol (encoded as a field element).
- Auxiliary columns for public inputs and bookkeeping.

Transition constraint. For each pair of states (i, j) and each symbol σ, the transition constraint asserts:

$$s'_j \;=\; \bigoplus_{i=0}^{n-1} s_i \otimes \phi(\mu_\sigma[i,j]) \;=\; \sum_{i=0}^{n-1} s_i \cdot \phi(\mu_\sigma[i,j]) \tag{5}$$

where the second equality holds because ϕ is a homomorphism. This is a standard inner-product constraint over $\mathbb{F}_p$.

Boundary constraints.

$$\text{Initial:} \quad s_i\big|_{t=0} = \phi(\alpha_i) \quad \text{for } i = 0, \ldots, n-1 \tag{6}$$

$$\text{Final:} \quad \text{output} = \sum_{i=0}^{n-1} s_i\big|_{t=T} \cdot \phi(\eta_i) \tag{7}$$

Theorem 5.2 (Tier 1 soundness)**.** *Let $\mathcal{A}$ be a WFA over an $\mathbb{F}_p$-embeddable semiring $\mathcal{S}$, and let $\mathsf{AIR}_{\mathcal{A}}$ be the AIR program produced by Tier 1 compilation. For every input $w \in \Sigma^*$, the unique accepting trace of $\mathsf{AIR}_{\mathcal{A}}$ on w encodes the value $\phi(\mathcal{A}(w))$ in the output column of the final row.*

Proof sketch. By induction on the length of w. The base case is Eq. (6). The inductive step follows from Eq. (5) and the homomorphism property (4): ϕ commutes with both $\oplus$ and $\otimes$, so the field-arithmetic constraint faithfully encodes the semiring transition. The final-row readout (Eq. (7)) computes ϕ of the semiring inner product, which equals $\phi(\mathcal{A}(w))$. □ □

The key property of Tier 1 is *genericity*: the same proof applies to *every* embeddable semiring. Adding a new embeddable semiring requires only defining ϕ; soundness is inherited automatically.

5.2 Tier 2: Gadget-Assisted Compilation

The tropical semiring $(\mathbb{Z} \cup \{+\infty\}, \min, +)$ is *not* $\mathbb{F}_p$-embeddable: the field has no min operation, and the additive identity $+\infty$ has no finite field representative. Tier 2 handles such semirings by introducing *gadgets*—small, hand-verified circuit fragments—that implement non-algebraic operations.

Comparison gadget. To compute $\min(a, b)$ over $\mathbb{F}_p$, we decompose a and b into their bit representations $a = \sum_{i=0}^{63} a_i \cdot 2^i$ and $b = \sum_{i=0}^{63} b_i \cdot 2^i$, enforce Boolean constraints $a_i(a_i - 1) = 0$ for each bit, and compute the less-than predicate via a carry chain:

$$\mathsf{lt}(a,b) = \begin{cases} 1 & \text{if } a < b \\ 0 & \text{otherwise} \end{cases} \qquad \min(a,b) = \mathsf{lt}(a,b) \cdot a + (1 - \mathsf{lt}(a,b)) \cdot b \tag{8}$$

The bit-decomposition adds $2 \times 64 = 128$ auxiliary columns per comparison.

Range check gadget. Ensures a value v lies in $[0, 2^k)$ by decomposing into k bits and asserting each is Boolean.

Per-semiring soundness. Unlike Tier 1, each gadget-assisted semiring requires its own soundness argument. The tropical semiring argument proceeds by showing that the comparison gadget correctly implements min over the integers (representable in $\mathbb{F}_p$ for values $< 2^{63}$), and that the tropical $\otimes = +$ is directly implemented by field addition.

Theorem 5.3 (Tier 2 soundness for tropical)**.** *Let $\mathcal{A}$ be a WFA over the tropical semiring, and let $\mathsf{AIR}_{\mathcal{A}}$ be the AIR program produced by Tier 2 compilation with comparison gadgets. For every input w with weight values bounded by 2^{63}, the unique accepting trace encodes $\mathcal{A}(w) \bmod p$ in the output column.*

5.3 Compilation Examples

We illustrate the two-tier strategy on three representative metrics.

Exact match → Boolean WFA → Tier 1. The exact-match WFA has $n + 1$ states for a reference of length n: state i indicates that the first i characters match. A single "dead" state absorbs mismatches. All transitions carry Boolean weights (0 or 1), embedded as field elements. The AIR has $2(n + 1) + 2$ columns and n rows.

ROUGE-L → tropical WFA → Tier 2. ROUGE-L computes the longest common subsequence (LCS) length, which is a tropical-semiring computation. The WFA has $O(nm)$ states for candidate length n and reference length m. Each transition involves a tropical min, compiled via the comparison gadget. The AIR has $2nm + 128c + O(1)$ columns, where c is the number of min operations per row.

BLEU-4 → 4 counting WFAs + gadgets. BLEU decomposes into four independent n-gram precision computations (one per $n \in \{1, 2, 3, 4\}$), each a counting-semiring WFA, plus an aggregation circuit that computes the clipped geometric mean and brevity penalty. The four WFA proofs are generated independently (Tier 1) and composed with the aggregation gadget (a small Tier 2 circuit). This decomposition keeps each individual proof small: the n-gram WFA for order k has $O(|\text{ref}|^k)$ states, but the reference is typically short enough that this is manageable.

6 Contamination Detection

Spectacles certifies not only that the score was computed correctly, but also that the test data was not leaked into the training corpus. We formalize this as a bound on n-gram overlap.

6.1 OPRF-Based PSI Protocol

Let L_1 be the set of n-grams extracted from the training corpus (held by the model provider) and L_2 the set of n-grams from the test set (held by the evaluator). The contamination detection protocol computes $|L_1 \cap L_2|$ without revealing the sets themselves, using an OPRF-based PSI protocol:

1. The evaluator sends OPRF-blinded versions of each n-gram in L_2 to the model provider.
2. The model provider evaluates the OPRF on each blinded element using its key k, returning the results.
3. The evaluator unblinds the results and compares against $\{F_k(x) : x \in L_2\}$ computed locally (using the provider's OPRF evaluations of L_1 received in a separate step).
4. The intersection cardinality is computed and compared to the threshold τ.

6.2 Trie-Structured Optimization

Standard flat-set PSI has communication complexity $O(|L_1| \cdot |L_2| \cdot \lambda)$ where λ is the security parameter. SPECTACLES exploits the hierarchical structure of n-grams: a 4-gram "`the cat sat on`" shares prefixes with the 3-gram "`the cat sat`" and the 2-gram "`the cat`".

We organize both L_1 and L_2 as tries, where each node represents an n-gram prefix. The OPRF evaluation proceeds level by level: if a prefix does not match, the entire subtree is pruned. This reduces communication to $O((n_1 + n_2) \cdot \lambda)$ where n_1, n_2 are the *trie sizes* (number of nodes), which are typically much smaller than $|L_1| \cdot |L_2|$ due to prefix sharing.

6.3 Threshold Mode

In threshold mode, the protocol produces a zero-knowledge proof that $|L_1 \cap L_2| < \tau$ without revealing the exact count. This is implemented via a comparison circuit that takes the PSI cardinality as a private input and outputs a single bit. The proof is composed with the evaluation certificate, yielding a single artifact that attests to both score correctness and contamination bounds.

6.4 Security Under the Commit-Then-Execute Framework

The PSI protocol operates under the semi-honest model. We address the concern that a malicious provider could deviate from the protocol to circumvent contamination detection.

Commitment binding. Before the PSI protocol begins, the model provider commits to its training n-gram set C_{train} via a binding commitment $h = H(\text{sort}(C_{\text{train}}) \| r)$ where H is a collision-resistant hash (BLAKE3) and r is a random nonce. The commitment is published before any PSI messages are exchanged. The verifier checks during the PSI protocol that the provider's inputs are consistent with the commitment.

Attack analysis. We analyze three attack vectors:

- *Input substitution*: The provider uses a different n-gram set than committed. This is prevented by the binding property of the commitment scheme: finding $C' \neq C_{\text{train}}$ with $H(C') = H(C_{\text{train}})$ requires breaking collision resistance.
- *Selective abort*: The provider aborts when overlap exceeds τ. The protocol requires completion for a valid certificate; an aborted protocol produces no attestation, and the absence of a certificate is itself informative.
- *Collusion*: A malicious provider colluding with the benchmark maintainer is outside our threat model (explicitly stated in §1).

Under these defenses, the semi-honest PSI suffices: the provider cannot substitute inputs (commitment binding) and cannot selectively abort (completion required). The remaining semi-honest assumption is that the provider follows the OPRF evaluation protocol, which is justified because deviations produce inconsistent OPRF outputs detectable by the verifier.

Limitations. We note three limitations of this analysis. First, the protocol does not achieve universally composable (UC) security; a full UC analysis with an ideal functionality specification is future work. Second, collusion between the model provider and the benchmark maintainer is outside our threat model. Third, without a verifiable OPRF (VOPRF), a sophisticated provider could selectively corrupt a small fraction of OPRF evaluations. Deploying a VOPRF (e.g., based on the Dodis–Yampolskiy VRF) would close this gap at the cost of additional computational overhead.

7 Metric Equivalence

Two implementations of the "same" metric may compute different functions due to implementation differences (tokenization, normalization, edge-case handling). SPECTACLES provides a decidable procedure for checking whether two EVALSPEC programs compute the same function.

Theorem 7.1 (Decidability of metric equivalence)**.** *For two WFAs $\mathcal{A}_1, \mathcal{A}_2$ over the same semiring $\mathcal{S}$ (where $\mathcal{S}$ is a field or a subsemiring of a field), $\mathcal{A}_1 = \mathcal{A}_2$ is decidable in time $O(n \log n)$ where $n = |Q_1| + |Q_2|$.*

Proof sketch. Minimize both WFAs using a weighted generalization of Hopcroft's partition-refinement algorithm [10, 4]. Two minimal WFAs over a field recognize the same formal power series if and only if they are isomorphic (up to a change of basis). Isomorphism of minimal WFAs is decidable by computing the canonical form and comparing. □ □

When $\mathcal{A}_1 \neq \mathcal{A}_2$, SPECTACLES generates a *distinguishing word* $w^* \in \Sigma^*$ such that $\mathcal{A}_1(w^*) \neq \mathcal{A}_2(w^*)$, providing a concrete counterexample.

Use case. An auditor can verify that two BLEU implementations (e.g., SacreBLEU and NLTK BLEU) compute the same function by expressing both in EVALSPEC, compiling to WFAs, and running the equivalence checker. If they disagree, the distinguishing word pinpoints the discrepancy.

8 Implementation

SPECTACLES is implemented in Rust, organized as a Cargo workspace. Table 3 reports the module breakdown.

8.1 Triple Implementation Pattern

Every metric is implemented three times:

1. **Reference implementation**: a direct, readable Rust implementation of the metric's mathematical definition.
2. **WFA implementation**: construction of the corresponding weighted automaton and evaluation via matrix–vector products.
3. **Circuit implementation**: generation of the AIR program and execution trace, with the score read from the final row.

All three implementations expose a common `TripleMetric` trait:

```
pub trait TripleMetric {
    type Input;
    type Score: PartialEq + Debug;
    fn score_reference(&self, input: &Self::Input) -> Self::Score;
    fn score_automaton(&self, input: &Self::Input) -> Self::Score;
    fn score_circuit(&self, input: &Self::Input) -> Self::Score;
}
```

Listing 2: The `TripleMetric` trait.

Table 3: Module breakdown of the SPECTACLES implementation.

Crate / Module	Files	LoC	Description
`spectacles-core`			
`wfa/`	9	28 738	Semiring definitions, WFA construction, operations, minimization, equivalence, field embedding, transducers
`circuit/`	9	42 555	Goldilocks field, AIR constraints, trace generation, gadgets, FRI, Merkle trees, STARK prover/verifier
`evalspec/`	7	22 920	Parser, type checker, semiring inference, denotational semantics, built-in metric definitions, compiler
`scoring/`	9	6 467	Reference implementations of all 7 metrics, tokenizer, differential testing harness
`psi/`	5	17 060	OPRF, trie, n-gram extraction, PSI protocol, commitment binding
`protocol/`	5	13 930	Commit–reveal state machine, certificates, transcripts, commitments
`utils/`	4	1 639	BLAKE3 hashing, serialization, math utilities
`spectacles-cli`	1	467	Command-line interface
`spectacles-integration`	2	487	End-to-end tests
`spectacles-examples`	4	527	Example programs
Total	**55**	**∼118 K**	

8.2 Differential Testing

The differential testing harness generates random (candidate, reference) pairs and asserts that all three implementations produce identical scores. Random pairs are generated by a seeded RNG to ensure reproducibility. The harness runs 100 K pairs per metric by default and reports any discrepancy with the failing input.

8.3 CLI Tool

The `spectacles-cli` crate provides the command-line interface:

```
# Score a single pair
spectacles score --metric bleu \
  --candidate "the cat sat" --reference "the cat sat on the mat"

# Run differential testing
spectacles differential-test --count 100000 --seed 42
```

Table 4: Differential testing results (100 K pairs per metric).

Metric	Pairs Tested	Agreements	Discrepancies
Exact Match	100 000	100 000	0
Token F1	100 000	100 000	0
BLEU	100 000	100 000	0
ROUGE-1	100 000	100 000	0
ROUGE-2	100 000	100 000	0
ROUGE-L	100 000	100 000	0
Regex Match	100 000	100 000	0
Pass@k	100 000	100 000	0
Total	**800 000**	**800 000**	**0**

```
# Compile a metric to an AIR circuit
spectacles compile-circuit --metric exact_match

# Estimate proof size
spectacles estimate-size --constraints 1024 --wires 64

# Batch scoring from JSONL
spectacles batch-score --input pairs.jsonl
```

Listing 3: CLI usage examples.

9 Evaluation

We evaluate Spectacles along five dimensions: differential testing correctness, proof size, circuit complexity, PSI correctness, and metric equivalence correctness.

9.1 Differential Testing

We generated 100 K random (candidate, reference) pairs per metric using a seeded random generator. Table 4 reports the results: all three implementations agree on every pair for every metric.

9.2 Circuit Complexity

Table 5 reports the AIR circuit characteristics per metric for typical input sizes.

9.3 Proof Size Estimates

STARK proof sizes depend on the trace dimensions and the security parameter λ. For $\lambda = 128$ bits of security and a blowup factor of 8, Table 6 provides estimates based on standard STARK proof-size formulae.

9.4 PSI Correctness

We tested the PSI contamination detector on synthetic corpora with known overlap. For each test, we constructed training and test n-gram sets with a predetermined intersection, ran the protocol,

Table 5: AIR circuit characteristics per metric (reference length $n = 32$, candidate length $m = 32$, vocabulary $|V| = 256$).

Metric	States	Columns	Rows	Constraints	Tier
Exact Match	33	68	32	33	1
Token F1	256	514	32	256	1
BLEU-1	32	66	32	32	1+G
BLEU-4	$4\times$	4×66	32	4×32	1+G
ROUGE-1	32	66	32	32	1
ROUGE-2	992	1 986	32	992	1
ROUGE-L	1 024	$2\,048+128c$	32	$1\,024+c$	2
Regex Match	$\leq 2^k$	$\leq 2^{k+1}$	m	$\leq 2^k$	1
Pass@k	kt	$2kt + 2$	t	kt	1

Table 6: Estimated STARK proof sizes per metric ($\lambda = 128$, blowup $= 8$).

Metric	Trace Size (KiB)	Proof Size (KiB)	Verification (ms)
Exact Match	17	45–85	< 5
Token F1	131	120–220	< 10
BLEU-4	67	90–170	< 10
ROUGE-1	17	45–85	< 5
ROUGE-2	507	250–450	< 15
ROUGE-L	1 200	400–750	< 20
Pass@k	varies	50–200	< 10

and verified that the reported cardinality matched the ground truth. Across 1 000 test cases with overlaps ranging from 0% to 100%, the protocol reported the correct cardinality in every case. The trie-structured optimization reduced communication by 3.2×–8.7× compared to flat-set PSI, with the largest gains on corpora with high prefix sharing (natural language text).

9.5 Metric Equivalence

We tested the equivalence checker by constructing pairs of WFAs that are either (a) semantically equivalent (same metric, different construction order) or (b) semantically distinct (e.g., BLEU with add-one smoothing vs. floor smoothing). The checker correctly identified all equivalent pairs and produced valid distinguishing words for all inequivalent pairs. Minimization ran in under 10 ms for all WFAs with $\leq$ 1,024 states.

9.6 Property-Based Axiom Verification

We verify that all Rust semiring implementations satisfy the algebraic axioms proven in the Lean 4 `KleeneSemiring` formalization. Using the `proptest` property-based testing framework, we test 38 properties across four semiring types (Boolean, Counting, Tropical, Goldilocks): additive commutativity and associativity, multiplicative associativity, distributivity, identity elements, and zero annihilation. All 38 properties pass on 256 random inputs each.

An independent Python test suite (`tests/test_semiring_axioms.py`) re-verifies the same ax-

Table 7: Pass@k arithmetization scaling.

k	t	States	Columns	Rows	Constraints
1	100	100	202	100	100
10	100	1 000	2 002	100	1 000
100	100	10 000	20 002	100	10 000
100	1 000	100 000	200 002	1 000	100 000

ioms outside the Rust ecosystem, including WFA equivalence testing via brute-force enumeration on all words up to length 6. All 22 Python tests pass on 500 random trials per axiom, providing cross-language validation.

9.7 Mathematical Verification

During a systematic verification pass, we identified and corrected two bugs in the Goldilocks field implementation:

1. A Montgomery inverse constant (`MONTGOMERY_INV`) was incorrect, which would have affected Montgomery-form arithmetic. The main reduction path was unaffected.
2. A redundant line in Lagrange interpolation corrupted the basis polynomial computation. The fix was a single-line deletion.

These corrections were verified by the property-based test suite and do not affect the differential testing results, which use the standard reduction path.

9.8 Pass@k Arithmetization Scaling

The pass@k metric [5] has WFA state complexity $O(kt)$ where k is the number of samples and t is the number of test cases. The circuit has $2kt + 2$ columns, t rows, and kt constraints. Table 7 characterizes the scaling for typical values.

For $k \leq 100$ and $t \leq 100$, the circuit is feasible with existing STARK provers (20K columns, comparable to ROUGE-L for long inputs). At $k = 100$, $t = 1{,}000$, the 200K-column circuit approaches current STARK capacity limits; batching or recursive proof composition would be needed for larger instances.

9.9 Comparison with Existing Frameworks

Spectacles addresses a fundamentally different problem from evaluation harnesses such as HELM [41] and CheckList [22]. HELM provides a standardized suite of benchmarks and metrics but offers no machine-checkable correctness guarantee—a bug in the scoring implementation silently corrupts all results. CheckList provides behavioral testing of NLP models but does not address score integrity or contamination. Spectacles is complementary: it provides cryptographic certificates that the score computed by any metric in its expressible fragment is correct and uncontaminated, regardless of the evaluation harness used. The two approaches compose naturally—one could run HELM's benchmark suite with Spectacles-certified scoring.

10 Related Work

Zero-knowledge proof systems. STARK [3] provides transparent (no trusted setup), post-quantum-secure proofs with polylogarithmic verification. SNARK systems such as Groth16 [9] achieve constant-size proofs but require a trusted setup and are not post-quantum secure. Circuit-level DSLs including Circom [1] and Noir [15] compile high-level programs to rank-1 constraint systems (R1CS); Spectacles targets AIR constraints, which are better suited to the iterative structure of automaton execution. EZKL [11] proves machine-learning inference in zero knowledge but operates at the neural-network level, not the evaluation-metric level.

Verified compilers. CompCert [13] provides a verified C compiler with a Coq proof of semantic preservation. CakeML [12] provides a verified ML compiler with an HOL4 proof. Spectacles shares the goal of semantic preservation but targets a different domain: compilation from a specification DSL to arithmetic circuits rather than from a general-purpose language to machine code. A Lean 4 formalization of the Spectacles compilation theorems is planned as future work.

NLP evaluation metrics. BLEU [16] measures n-gram precision with a brevity penalty. ROUGE [14] measures recall-oriented n-gram overlap and longest common subsequence. Token F1 computes the harmonic mean of token-level precision and recall. Pass@k [5] estimates the probability that at least one of k code samples passes a test suite. Post [17] documents the sensitivity of BLEU to implementation details. All of these metrics are expressible in EvalSpec.

Verifiable computation for ML. VerifiableEvals [22] uses zero-knowledge proofs to certify that an evaluation script was executed correctly on committed data. However, it proves program execution, not specification conformance: a buggy script produces a valid proof of a wrong answer. Spectacles addresses this by compiling from a *specification* (the EvalSpec definition) rather than from an *implementation*, and by providing metric equivalence checking to ensure specification fidelity. ZKML [11] focuses on proving neural network inference, which is complementary to evaluation certification.

Weighted automata theory. Schützenberger [20] introduced the theory of formal power series recognized by finite automata. Berstel and Reutenauer [4] provide a comprehensive treatment including minimization and equivalence. Droste, Kuich, and Vogler [6] survey the modern theory. Spectacles applies this classical theory to a new domain: compiling evaluation metrics to zero-knowledge circuits.

Contamination detection. Sainz et al. [19] propose statistical tests for benchmark contamination. Shi et al. [21] detect contamination via membership inference attacks on n-grams. Both approaches are statistical and cannot certify individual evaluations. Spectacles provides cryptographic contamination certificates via PSI, which offers simulation-secure guarantees rather than statistical confidence intervals.

11 Conclusion and Future Work

We presented Spectacles, the first system that produces contamination-certified evaluation certificates for NLP benchmarks. By observing that standard scoring functions decompose into weighted finite automata over semirings, Spectacles bridges algebraic automata theory and zero-knowledge

cryptography, compiling EvalSpec specifications to STARK arithmetic circuits via a two-tier architecture. The resulting certificates prove both score correctness and training-data separation in a single machine-checkable artifact.

Future work. The primary gap in the current system is the absence of verified extraction from the Lean 4 proofs to the Rust implementation. While property-based testing and differential testing provide strong empirical evidence of correspondence, a machine-checked extraction pipeline (akin to CakeML's HOL4-to-ML extraction) would close the verification gap entirely. We plan to formalize the compilation theorems in Lean 4, providing a machine-checked proof of Theorems 5.2 and 5.3, and to prove Hopcroft minimization correctness (currently deferred with differential testing coverage). We aim to extend EvalSpec to metrics that require continuous embeddings (e.g., BERTScore [23]), which will require integrating ZKML techniques for the embedding computation with WFA-based scoring. Finally, we intend to provide a universally composable (UC) security analysis of the full protocol, establishing that the contamination-certified certificate is simulation-secure under standard cryptographic assumptions.

References

[1] J. Belles-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina. Circom: A robust and scalable language for building complex zero-knowledge circuits. In *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2022.

[2] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. In *Proc. ICALP*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl, 2018.

[3] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, Report 2018/046, 2018.

[4] J. Berstel and C. Reutenauer. *Noncommutative Rational Series with Applications*. Cambridge University Press, 2011.

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[6] M. Droste, W. Kuich, and H. Vogler, editors. *Handbook of Weighted Automata*. Springer, 2009.

[7] M. J. Freedman, K. Nissim, and B. Pinkas. Efficient private matching and set intersection. In *Proc. EUROCRYPT*, volume 3027 of *LNCS*, pages 1–19. Springer, 2004.

[8] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[9] J. Groth. On the size of pairing-based non-interactive arguments. In *Proc. EUROCRYPT*, volume 9666 of *LNCS*, pages 305–326. Springer, 2016.

[10] J. E. Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. In Z. Kohavi and A. Paz, editors, *Theory of Machines and Computations*, pages 189–196. Academic Press, 1971.

[11] J. Kang, H. Sun, H. Gu, and C. Zhuo. Scaling ZKML: Efficient zero-knowledge proofs for neural network inference. *arXiv preprint arXiv:2406.04296*, 2024.

[12] R. Kumar, M. O. Myreen, M. Norrish, and S. Owens. CakeML: A verified implementation of ML. In *Proc. POPL*, pages 179–191. ACM, 2014.

[13] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[14] C.-Y. Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proc. ACL Workshop*, pages 74–81, 2004.

[15] Aztec Labs. Noir: A domain-specific language for zero-knowledge proofs. https://noir-lang.org/, 2023.

[16] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proc. ACL*, pages 311–318, 2002.

[17] M. Post. A call for clarity in reporting BLEU scores. In *Proc. WMT*, pages 186–191, 2018.

[18] G. Rusu and J. Rutten. Coinductive stream calculus and automata theory. Technical Report SEN-R0024, CWI, 2000.

[19] O. Sainz, J. Campos, I. García-Ferrero, J. Etxaniz, O. Lopez de Lacalle, and E. Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of EMNLP*, pages 10776–10787, 2023.

[20] M.-P. Schützenberger. On the definition of a family of automata. *Information and Control*, 4(2–3):245–270, 1961.

[21] W. Shi, A. Ajith, M. Sha, A. Jacobs, Y. Fang, C. Liu, Y. Bisk, and G. Neubig. Detecting pretraining data from large language models. In *Proc. ICLR*, 2024.

[22] T. South, A. Mavroudis, and S. Hoyer. VerifiableEvals: Verifiable evaluation of language models using zero-knowledge proofs. *arXiv preprint arXiv:2410.15113*, 2024.

[23] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi. BERTScore: Evaluating text generation with BERT. In *Proc. ICLR*, 2020.

[24] O. Goldreich. *Foundations of Cryptography: Volume 1, Basic Tools*. Cambridge University Press, 2001.

[25] M. Blum, P. De Santis, S. Micali, and G. Persiano. Noninteractive zero-knowledge. *SIAM Journal on Computing*, 20(6):1084–1118, 1991.

[26] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, and Y. Ishai. Verifiable random functions from standard assumptions. In *Proc. TCC*, volume 10677 of *LNCS*, pages 336–362. Springer, 2017.

[27] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, Report 2019/953, 2019.

[28] S. Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Proc. CRYPTO*, volume 12172 of *LNCS*, pages 704–737. Springer, 2020.

[29] C. Allauzen, M. Riley, J. Schalkwyk, W. Skut, and M. Mohri. OpenFst: A general and efficient weighted finite-state transducer library. In *Proc. CIAA*, volume 4783 of *LNCS*, pages 11–23. Springer, 2007.

[30] M. Mohri. Weighted automata algorithms. In *Handbook of Weighted Automata*, pages 213–254. Springer, 2009.

[31] C. Gentry. A fully homomorphic encryption scheme. PhD thesis, Stanford University, 2009.

[32] R. Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *Proc. FOCS*, pages 136–145. IEEE, 2001.

[33] D. Kozen. Automata and computability. *Undergraduate Texts in Computer Science*. Springer, 1997.

[34] A. Sahai and B. Waters. Fuzzy identity-based encryption. In *Proc. EUROCRYPT*, volume 3494 of *LNCS*, pages 457–473. Springer, 2005.

[35] J. Kilian. A note on efficient zero-knowledge proofs and arguments. In *Proc. STOC*, pages 723–732. ACM, 1992.

[36] S. Micali. Computationally sound proofs. *SIAM Journal on Computing*, 30(4):1253–1298, 2000.

[37] J. Thaler. *Proofs, Arguments, and Zero-Knowledge*. Foundations and Trends in Privacy and Security, 2022.

[38] C. Weng, K. Yang, J. Katz, and X. Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *Proc. IEEE S&P*, pages 1074–1091, 2021.

[39] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *Proc. IEEE S&P*, pages 315–334, 2018.

[40] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In *Proc. EUROCRYPT*, volume 12105 of *LNCS*, pages 738–768. Springer, 2020.

[41] J. Lee, D. Mukherjee, H. W. Chung, T. Berg-Kirkpatrick, and R. Salakhutdinov. Decompiling LLM pre-training data. *arXiv preprint arXiv:2307.16789*, 2023.

[42] S. Goldfeder, H. Kalodner, D. Reisman, and A. Narayanan. When the cookie meets the blockchain: Privacy risks of web payments via cryptocurrencies. *Proceedings on Privacy Enhancing Technologies*, 2018(4):179–199, 2018.

[43] Y. Ishai, J. Kilian, K. Nissim, and E. Petrank. Extending oblivious transfers efficiently. In *Proc. CRYPTO*, volume 2729 of *LNCS*, pages 145–161. Springer, 2003.

[44] M. Chase and P. Miao. Private set intersection in the Internet setting from lightweight oblivious PRF. In *Proc. CRYPTO*, volume 12172 of *LNCS*, pages 34–63. Springer, 2020.

[45] C. Meadows. A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party. In *Proc. IEEE S&P*, pages 134–137, 1986.

A Detailed Proof of Tier 1 Soundness

Full proof of Theorem 5.2. Let $\mathcal{A} = (Q, \Sigma, \{\mu_\sigma\}, \alpha, \eta)$ be a WFA over an $\mathbb{F}_p$-embeddable semiring $\mathcal{S}$ with embedding $\phi : \mathcal{S} \to \mathbb{F}_p$, and let $w = \sigma_1 \cdots \sigma_T$.

We prove by induction on t that after processing $\sigma_1 \cdots \sigma_t$, the state vector in the trace satisfies $s_i|_t = \phi(v_i^{(t)})$ for all $i \in \{0, \ldots, |Q| - 1\}$, where $v^{(t)} = \alpha \otimes \mu_{\sigma_1} \otimes \cdots \otimes \mu_{\sigma_t}$.

Base case ($t = 0$): The boundary constraint (Eq. (6)) sets $s_i|_0 = \phi(\alpha_i)$, and $v^{(0)} = \alpha$, so $s_i|_0 = \phi(v_i^{(0)})$. ✓

Inductive step: Assume $s_i|_t = \phi(v_i^{(t)})$ for all i. The transition constraint (Eq. (5)) asserts:

$$s_j|_{t+1} = \sum_{i=0}^{|Q|-1} s_i|_t \cdot \phi(\mu_{\sigma_{t+1}}[i, j])$$

By the inductive hypothesis:

$$= \sum_{i=0}^{|Q|-1} \phi(v_i^{(t)}) \cdot \phi(\mu_{\sigma_{t+1}}[i, j])$$

Since ϕ is a homomorphism ($\phi(a \otimes b) = \phi(a) \cdot \phi(b)$ and $\phi(a \oplus b) = \phi(a) + \phi(b)$):

$$= \phi\left(\bigoplus_{i=0}^{|Q|-1} v_i^{(t)} \otimes \mu_{\sigma_{t+1}}[i, j]\right) = \phi(v_j^{(t+1)})$$

which is the (j)-th component of the new state vector. ✓

Final readout: At $t = T$, the output column contains:

$$\text{output} = \sum_{i=0}^{|Q|-1} s_i|_T \cdot \phi(\eta_i) = \sum_i \phi(v_i^{(T)}) \cdot \phi(\eta_i) = \phi\left(\bigoplus_i v_i^{(T)} \otimes \eta_i\right) = \phi(\mathcal{A}(w))$$

□

B Boolean Embedding Validity

The Boolean semiring $(\{0, 1\}, \vee, \wedge, 0, 1)$ embeds into $\mathbb{F}_p$ via $\phi(0) = 0$, $\phi(1) = 1$. This embedding satisfies the multiplicative homomorphism ($\phi(a \wedge b) = \phi(a) \cdot \phi(b)$) and identity preservation. However, $\vee$ does not map to field addition in general: $\phi(1 \vee 1) = \phi(1) = 1 \neq 2 = \phi(1) + \phi(1)$.

The embedding is nevertheless valid for *deterministic* Boolean WFA execution. In a deterministic WFA, for each state q and input symbol σ, at most one outgoing transition has weight 1 (all others have weight 0). Consequently, the inner sum $\bigoplus_i v_i \otimes \mu_\sigma[i, j] = \bigvee_i (v_i \wedge \mu_\sigma[i, j])$ has at most one non-zero term, so $\vee$ is never applied to two 1-values simultaneously. Under this restriction, $\phi(\bigoplus_i \cdots) = \sum_i \phi(\cdots)$ holds because the sum contains at most one non-zero summand.

C Hopcroft Minimization Testing

The Hopcroft minimization algorithm in `wfa/minimization.rs` is verified by the following testing methodology:

1. **Known-minimal automata**: We construct WFAs that are provably minimal (e.g., a 3-state DFA for a single-character pattern) and verify that the minimization algorithm does not reduce the state count.
2. **Redundant automata**: We construct WFAs with deliberately duplicated states (doubling the state space) and verify that minimization recovers the original state count.
3. **Brute-force equivalence**: For both original and minimized WFAs, we enumerate all strings up to length 6 and verify that the weight functions agree on every string. This is feasible for automata with ≤ 8 states over alphabets of size ≤ 4 (at most $\sum_{k=0}^{6} 4^k = 5{,}461$ strings).
4. **Randomized testing**: We generate random WFAs with 2–5 states and verify self-equivalence, catching any bugs in the evaluation engine.

The Lean 4 proof of Hopcroft minimization correctness is deferred as future work. The current testing methodology provides high confidence but not machine-checked certainty.